



Informe Técnico de Programación Full Stack

Maquetado del Sitio Web Mobile First — Primera Entrega

Solicitante: I.T.S. – Instituto Tecnológico Superior Arias – Balparda

Nombre de fantasía (empresa): Zenko

Aplicación web: Sportana

Grupo: 3.º BT Tecnologías de la Información

Turno: Matutino

Unidad Curricular: Programación Full Stack

Integrantes del grupo:

Albino, Tomás — Díaz, Thiago — Scoppe, Andrés (referente) — Silva, Federico

Fecha de entrega: 27 / 07 / 2026

Instituto Tecnológico Superior Arias Balparda.
Blvr. José Batlle y Ordóñez 3570 esq. Gral. Flores — Montevideo.

1. Índice

El presente índice refleja la paginación de esta versión del documento. Si se agregan o quitan contenidos, los números de página deberán revisarse antes de la entrega final.

2. Vocabulario y Simbología	3
3. Desarrollo del tema	4
3.1 Introducción	4
3.2 Metodología de maquetado	4
3.2.1 Enfoque Mobile First	4
3.2.2 Flexbox como modelo de layout	5
3.2.3 Sistema de breakpoints	6
3.3 Arquitectura de la maqueta	6
3.3.1 Organización de archivos	6
3.3.2 Sistema de diseño visual	7
3.3.3 Tipografía	8
3.3.4 Iconografía	8
3.4 Páginas maquetadas y sus funcionalidades	8
3.4.1 Inicio (index.html)	8
3.4.2 Torneos (torneos.html)	9
3.4.3 Deportes (deportes.html)	9
3.4.4 Detalle de torneo (detalle.html)	9
3.4.5 Creación de torneo — formulario en tres pasos	9
3.4.6 Perfil (perfil.html)	10
3.4.7 Autenticación	10
3.5 Capturas de pantalla	10
3.6 Accesibilidad y buenas prácticas	14
3.7 Elementos HTML y propiedades CSS utilizados	15
3.7.1 Etiquetas HTML	15
3.7.2 Propiedades CSS	17
3.8 Conclusión general	20
4. Bibliografía	20
Hoja Testigo	21

2. Vocabulario y Simbología

A continuación se definen los principales términos técnicos utilizados a lo largo de este documento.

- **HTML5:** versión actual del lenguaje de marcado estándar para estructurar contenido web, que incorpora elementos semánticos como header, section, article y footer.
- **CSS3:** conjunto de reglas de estilo que definen la presentación visual de un documento HTML: color, tipografía, espaciado y disposición de los elementos en pantalla.
- **Flexbox:** modelo de layout de una dimensión de CSS que permite distribuir el espacio entre los elementos de un contenedor, alineándolos en fila o en columna.
- **CSS Grid:** modelo de layout de dos dimensiones de CSS que permite posicionar elementos simultáneamente en filas y columnas.
- **Mobile First:** metodología de diseño que define primero los estilos para pantallas pequeñas, ampliando progresivamente el diseño para pantallas mayores mediante media queries.
- **Media query:** regla de CSS que aplica un bloque de estilos únicamente cuando se cumple una condición sobre las características del dispositivo, como el ancho de pantalla.
- **Breakpoint:** ancho de pantalla a partir del cual el diseño cambia su disposición mediante una media query.
- **Diseño responsive:** enfoque de diseño web que permite que un mismo sitio se visualice correctamente en dispositivos de distintos tamaños.
- **Maquetado:** construcción de la estructura visual y de contenido de una interfaz, previa a su integración con lógica de servidor o base de datos.
- **Elemento semántico:** etiqueta HTML que describe el significado de su contenido (por ejemplo header, section o article), a diferencia de etiquetas genéricas como div o span.
- **Atributo alt:** texto alternativo de una imagen, utilizado por lectores de pantalla y mostrado en caso de que la imagen no pueda cargarse.
- **Viewport:** área visible de una página web dentro de la ventana o pantalla del dispositivo que la muestra.

- **Font Awesome:** biblioteca de iconos vectoriales utilizada mediante clases CSS, sin necesidad de archivos de imagen individuales por ícono.
- **Componente reutilizable:** fragmento de interfaz (por ejemplo una tarjeta o un botón) diseñado con una estructura y un estilo consistentes para repetirse en distintas partes del sitio.

3. Desarrollo del tema

3.1 Introducción

El presente documento corresponde a la primera entrega de la unidad curricular Programación Full Stack, en el marco del proyecto Sistema de Gestión Deportiva Modular (SGDM), desarrollado por la empresa Zenko bajo el nombre comercial Sportana. La letra de proyecto establece, para esta primera entrega, el maquetado completo del sitio siguiendo la filosofía Mobile First, utilizando Flexbox o Grid como modelo de layout.

Este informe describe el trabajo realizado sobre dicho maquetado: la metodología aplicada, la organización de los archivos, el sistema de diseño visual definido, las páginas construidas y las funcionalidades de interfaz que cada una implementa, así como las decisiones técnicas tomadas y su fundamento. Se aclara que, tratándose de la primera entrega, el sitio se encuentra maquetado con datos de ejemplo y no cuenta todavía con lógica de servidor, base de datos ni comunicación asincrónica, aspectos que se incorporarán en las siguientes entregas.

3.2 Metodología de maquetado

3.2.1 Enfoque Mobile First

La letra de proyecto exige, para esta primera entrega, que el maquetado se realice bajo la filosofía Mobile First: no se trata de una alternativa entre otras, sino de un requisito puntual de la asignatura. Más allá del carácter obligatorio de este criterio, el equipo coincide con dicha exigencia y la considera acertada para el caso particular de Sportana, por los motivos que se detallan a continuación.

Bajo el enfoque Mobile First, siguiendo la definición original de Wroblewski (2011), los estilos base de cada elemento se piensan primero para pantallas pequeñas, y las reglas para pantallas progresivamente más grandes se agregan mediante media queries de tipo min-width, en lugar de partir de un diseño de escritorio y adaptarlo luego hacia abajo con max-width.

Buena parte de los usuarios de SGDM —un organizador consultando el estado de un torneo desde la cancha, un participante revisando su próximo enfrentamiento— acceden con frecuencia desde un teléfono, muchas veces durante el desarrollo del propio evento deportivo. Priorizar la experiencia mobile evita que ese caso de uso quede como una adaptación de segundo orden de un diseño pensado originalmente para escritorio, motivo por el cual el equipo entiende que el requisito de la letra se ajusta bien a la realidad de uso del sitio.

La revisión del código confirma la aplicación consistente de este criterio: las 57 reglas @media presentes en las ocho hojas de estilo del proyecto utilizan exclusivamente la condición min-width, sin ninguna aparición de max-width. Esto indica que, en la totalidad de los casos, el punto de partida de cada estilo es la versión para pantallas pequeñas.

3.2.2 Flexbox como modelo de layout

El proyecto utiliza Flexbox como modelo de layout excluyente: la propiedad display:flex (o inline-flex) aparece 89 veces en el código, mientras que no se registra ningún uso de display:grid. Esta elección responde a la naturaleza del contenido a disponer: la gran mayoría de las secciones del sitio son listas de elementos similares en una dimensión —una fila de pestañas de filtro, una columna de campos de formulario, una grilla de tarjetas de torneos que pasa de una columna en mobile a cuatro en escritorio—, casos donde Flexbox resuelve el problema de forma directa mediante las propiedades flex-direction y flex-wrap.

CSS Grid hubiera resultado preferible si el sitio requiriera posicionar elementos en dos dimensiones de forma simultánea, por ejemplo un panel con áreas fijas superpuestas. No es el caso de las pantallas actuales de Sportana, por lo que se optó por mantener un único modelo de layout en todo el proyecto en lugar de combinar ambos sin necesidad. Se prevé evaluar la incorporación de Grid si una futura sección de estadísticas o un panel administrativo lo justifican.

3.2.3 Sistema de breakpoints

Se definieron cuatro breakpoints, aplicados de forma consistente en las ocho hojas de estilo: 480px, 768px, 1024px y 1280px. Estos valores corresponden, respectivamente, a teléfonos en orientación horizontal, tablets, notebooks y monitores de escritorio, siguiendo categorías de dispositivo habituales en lugar de atarse al ancho de un modelo puntual. Utilizar puntos de quiebre estándar facilita que el mismo criterio se extienda a nuevas páginas a medida que el proyecto avance.

3.3 Arquitectura de la maqueta

3.3.1 Organización de archivos

El maquetado se organiza en 12 documentos HTML y 8 hojas de estilo CSS, según se detalla en la Tabla 1. En lugar de concentrar todas las reglas en un único archivo, se separó una hoja base.css con los estilos compartidos por todo el sitio —encabezado, pie de página, tarjetas, botones, formularios genéricos— de siete hojas específicas por página, cada una con las reglas propias de esa vista únicamente.

Archivo	Líneas	Contenido
base.css	1052	Encabezado, pie de página, tarjetas, botones y formularios compartidos por todo el sitio
torneos.css	81	Filtro de búsqueda, pestañas de estado y grilla de torneos
deportes.css	300	Catálogo de disciplinas, tarjetas de deporte y sección de reglamentos
detalle.css	190	Vista de un torneo puntual: portada, resumen, pestañas y tabla de posiciones
crear.css	197	Formulario de creación de torneo en sus tres pasos
perfil.css	127	Cabecera de perfil, estadísticas y torneos del usuario
auth.css	158	Pantallas de inicio de sesión y registro (participante y federación)
inicio.css	154	Hero de portada, estadísticas generales y torneos destacados

Tabla 1. Hojas de estilo del proyecto, cantidad de líneas y contenido que regulan.

Esta separación evita que una página cargue reglas irrelevantes para su contenido y limita el impacto de un cambio: modificar el filtro de búsqueda de torneos, por ejemplo, solo requiere editar torneos.css, sin riesgo de afectar el formulario de creación de un torneo.

3.3.2 Sistema de diseño visual

La interfaz utiliza un esquema de color oscuro. La Tabla 2 resume la paleta relevada directamente del código, junto con la cantidad de veces que aparece cada color.

Color (hex)	Uso	Apariciones en el CSS
#8B5CF6	Violeta — color de marca	78
#FFFFFF	Blanco — texto principal	34
#B5B5C3	Gris claro — texto secundario	27
#2A2A36	Gris oscuro — bordes y divisores	23
#7C3AED	Violeta oscuro — estado hover	13
#6B6B78	Gris medio — íconos e indicadores	13
#0c0c1c	Negro azulado — fondo general de la página	7
#14141c	Negro azulado — fondo de tarjetas	6
#181821	Negro azulado — fondo de elementos secundarios	6
#12121A	Negro azulado — fondo de tarjetas de estadísticas	5
#F5C542	Dorado — primera posición o destacado	1
#CD7F32	Bronce — tercera posición	1
#EF4444	Rojo — estados de error o cancelación	1

Tabla 2. Paleta de colores identificada en las hojas de estilo del proyecto.

Los distintos tonos de negro azulado utilizados como fondo (#0c0c1c, #12121A, #14141c, #181821) dan profundidad a la interfaz sin recurrir a un único negro plano, mientras que el violeta (#8B5CF6)

actúa como color de marca en botones primarios, enlaces activos y distintivos de estado. Un violeta más oscuro (#7C3AED) se reserva para los estados hover, siguiendo la convención habitual de oscurecer el color de base al interactuar con un elemento. Se identificaron además colores semánticos puntuales —dorado y bronce para indicadores de posición, rojo para estados de error— coherentes con las convenciones de semáforo de estado más frecuentes en interfaces web.

3.3.3 Tipografía

La tipografía definida en el CSS es Poppins, declarada junto con una pila de alternativas del sistema (-apple-system, BlinkMacSystemFont, sans-serif) para los casos en que no esté disponible. Al revisar el código se constató que, en el estado actual del maquetado, Poppins no se encuentra vinculada mediante una hoja de Google Fonts ni un @font-face local, por lo que el sitio se renderiza actualmente con la fuente del sistema operativo del usuario. Se deja pendiente la incorporación del enlace correspondiente para la próxima entrega, una vez definida la estrategia de carga de recursos externos del proyecto.

3.3.4 Iconografía

El proyecto utiliza Font Awesome 6.7.2 (Font Awesome, 2026) mediante su hoja de estilos alojada en CDN, aplicando los íconos a través de clases (por ejemplo fa-solid fa-magnifying-glass para el buscador). Esta decisión evita mantener un conjunto propio de archivos SVG o de imagen para cada ícono utilizado en filtros, botones y tarjetas del sitio.

El enlace exacto incluido en el <head> de las doce páginas del sitio es:

<https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.7.2/css/all.min.css>

3.4 Páginas maquetadas y sus funcionalidades

Se detallan a continuación las doce páginas construidas en esta entrega, agrupadas según la función que cumplen dentro del sitio.

3.4.1 Inicio (index.html)

La página de inicio presenta el hero principal del sitio, con el título “Gestiona tus torneos de manera simple y profesional”, dos llamados a la acción (Crear torneo, Explorar torneos) y cuatro tarjetas de estadísticas (deportes disponibles, torneos creados, usuarios registrados, torneos activos). Debajo se ubica una sección de torneos destacados.

El hero utiliza una imagen de fondo a pantalla completa mediante `background-size: cover`, mientras que el contenido (título, subtítulo, botones y estadísticas) se aloja en un contenedor interno independiente de esa imagen, de forma de poder acotar el ancho del texto sin recortar el fondo decorativo.

3.4.2 Torneos (torneos.html)

Implementa la exploración y el filtrado de torneos: un buscador por texto, tres selectores (deporte, módulo de competencia, fecha) agrupados en una tarjeta con borde propio, cuatro pestañas de estado (todos, inscripciones abiertas, próximos, destacados) con su contador correspondiente, y la grilla de tarjetas de torneo.

La grilla se maqueta en columna en mobile y pasa a fila con ajuste automático (`flex-wrap`) a partir de los 768px, mostrando dos, tres o cuatro tarjetas por fila según el ancho disponible, mediante los anchos `calc(50% - 10px)`, `calc(33.333% - 14px)` y `calc(25% - 15px)` en cada breakpoint sucesivo.

3.4.3 Deportes (deportes.html)

Funciona como catálogo de disciplinas soportadas por la plataforma. Cada tarjeta muestra una imagen representativa, el nombre de la disciplina, el organismo que regula su reglamento (IFAB/FIFA para fútbol, FIBA para básquetbol, FIDE para ajedrez, entre otros) y una breve descripción, junto con un enlace para consultar el reglamento completo. La estructura del filtro replica la de torneos.html, manteniendo el mismo patrón de interacción en ambas páginas.

3.4.4 Detalle de torneo (detalle.html)

Muestra la información completa de un torneo puntual: portada, estado de inscripción, organizador, fechas, cupos y reglamento, junto con pestañas para consultar el calendario de enfrentamientos y la tabla de posiciones. A partir de los 1024px, la portada y el resumen se disponen en fila mediante `flex-direction: row`; en mobile se apilan verticalmente.

3.4.5 Creación de torneo — formulario en tres pasos (crear.html, crear2.html, crear3.html)

La creación de un torneo se maqueta como un formulario dividido en tres pasos, con un indicador visual que marca el paso actual,

los completados y los pendientes mediante las clases activo y hecho. El primer paso releva los datos generales (nombre, organizador, deporte) y el módulo de competencia; el segundo, la ubicación, las fechas y el reglamento; el tercero, los cupos de participantes y una vista previa de cómo se verá la tarjeta del torneo una vez publicado.

Dividir la carga de datos en pasos, en lugar de un único formulario extenso, reduce la percepción de esfuerzo para quien crea el torneo y permitirá, en la etapa de integración con el servidor, validar cada bloque de información de forma independiente.

3.4.6 Perfil (perfil.html)

Presenta los datos del usuario autenticado (nombre, categoría, avatar), sus estadísticas de participación y el listado de torneos en los que participa, reutilizando el mismo patrón de pestañas y tarjetas que las demás páginas del sitio.

3.4.7 Autenticación (login.html, login-federacion.html, registro.html, registro-federacion.html)

El acceso al sistema contempla dos flujos de registro diferenciados: registro.html, para participantes individuales (nombre, apellido, correo, usuario, contraseña), y registro-federacion.html, para organizaciones deportivas (nombre de la federación, deporte que representa, RUT, representante legal, correo institucional, teléfono de contacto). La separación responde a que una federación necesita acreditar datos institucionales para ser habilitada como organizadora de torneos, información que un participante individual no debe completar. login.html y login-federacion.html siguen el mismo criterio de separación para el inicio de sesión.

3.5 Capturas de pantalla

Se presenta a continuación evidencia visual del maquetado, contrastando la vista mobile (375px de ancho, resolución de referencia de un teléfono habitual) con la vista de escritorio (1440px), a fin de evidenciar la aplicación práctica del enfoque Mobile First descrito en la sección 3.2.1.

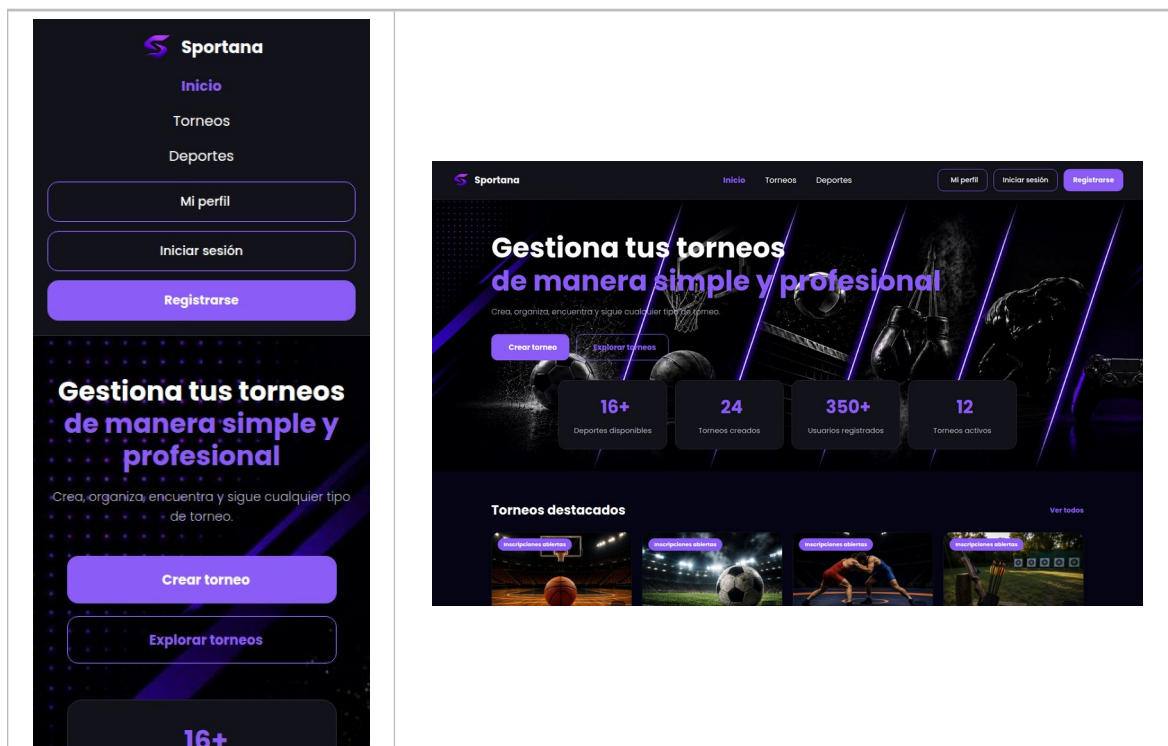


Figura 1. Inicio — vista mobile (375px) y desktop (1440px). El hero conserva la imagen de fondo completa en ambos anchos; el contenido de texto se recentra y las estadísticas pasan de columna a fila.

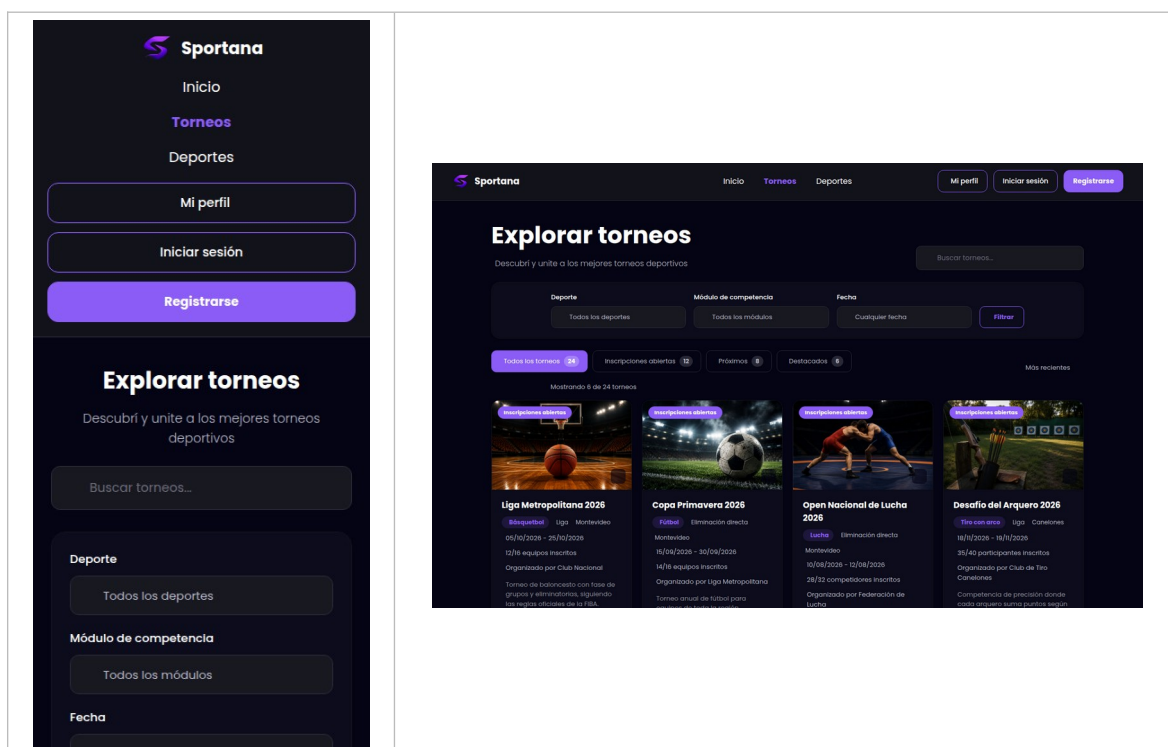


Figura 2. Torneos — vista mobile (375px) y desktop (1440px). El filtro pasa de columna a fila y la grilla de tarjetas pasa de una a cuatro columnas entre mobile y escritorio.

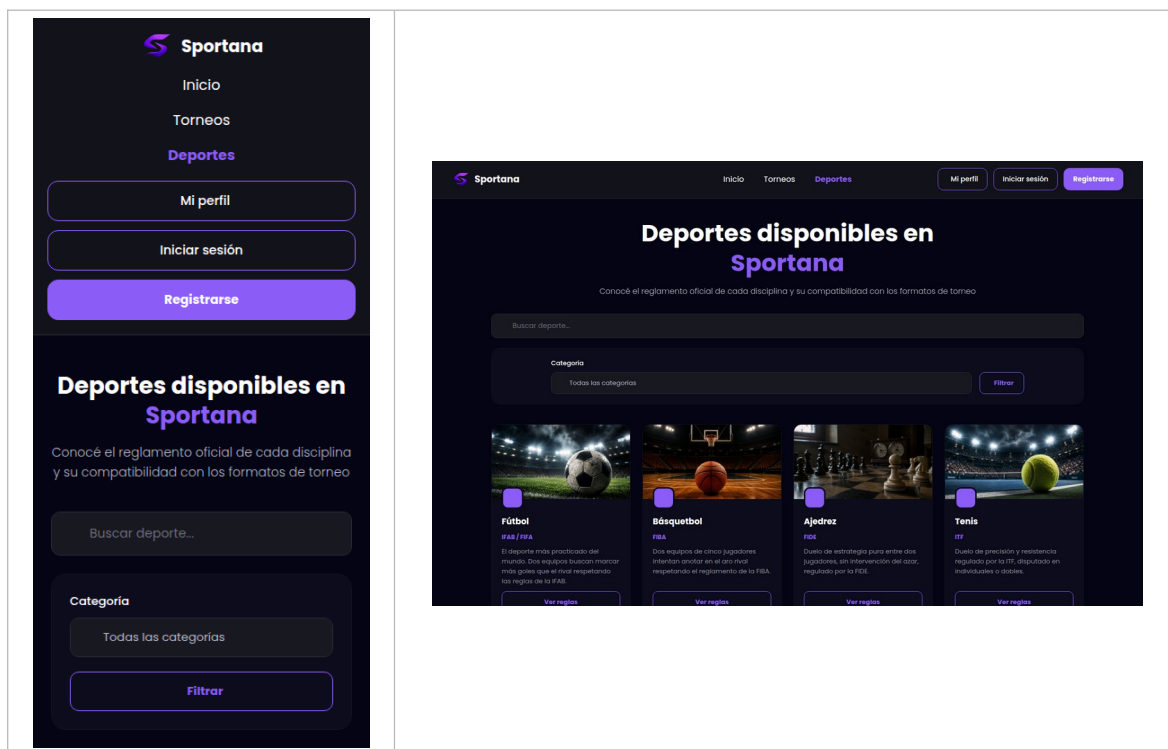


Figura 3. Deportes — vista mobile (375px) y desktop (1440px). El catálogo de disciplinas replica el mismo comportamiento de grilla que la página de torneos.

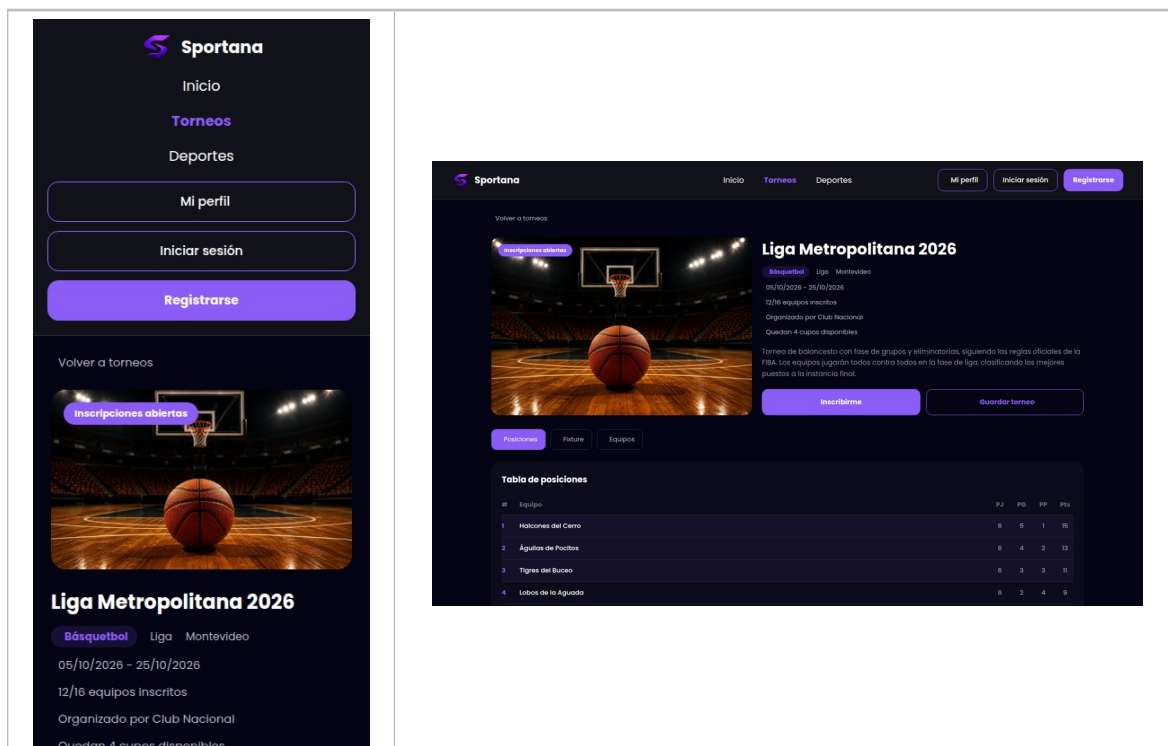
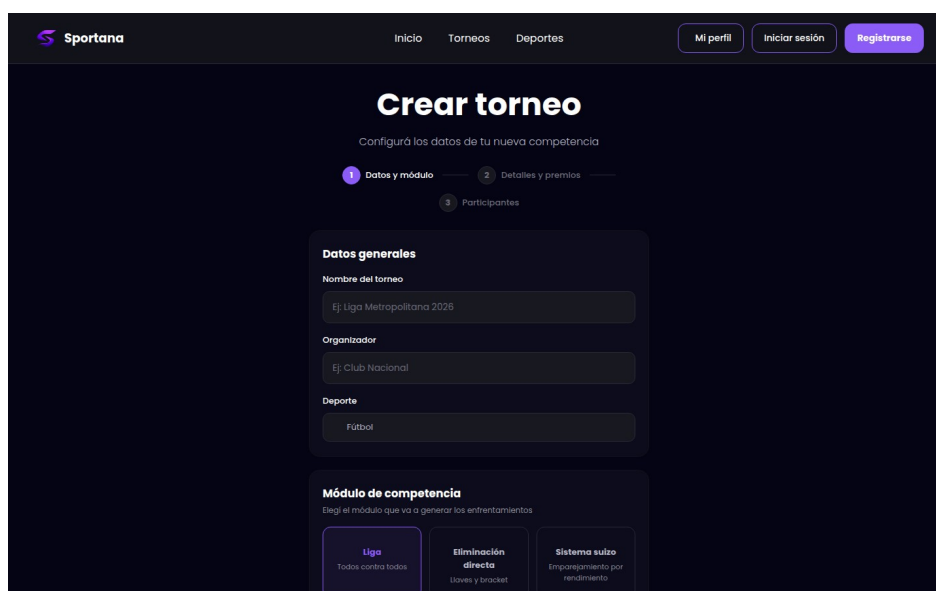


Figura 4. Detalle de torneo — vista mobile (375px) y desktop (1440px). La portada y el resumen pasan de apilados (mobile) a dispuestos en fila (escritorio).



Crear torneo

Configurá los datos de tu nueva competencia

1 Datos y módulo — 2 Detalles y premios — 3 Participantes

Datos generales

Nombre del torneo
Ej: Liga Metropolitana 2026

Organizador
Ej: Club Nacional

Deporte
Fútbol

Módulo de competencia
Elegí el módulo que va a generar los enfrentamientos

☒ **Liga**
 Todos contra todos

☐ **Eliminación directa**
 Llaves y bracket

☐ **Sistema suizo**
 Emparejamiento por rendimiento

Figura 5. Creación de torneo (paso 1 de 3). Formulario de datos generales, con el indicador de pasos en la parte superior.

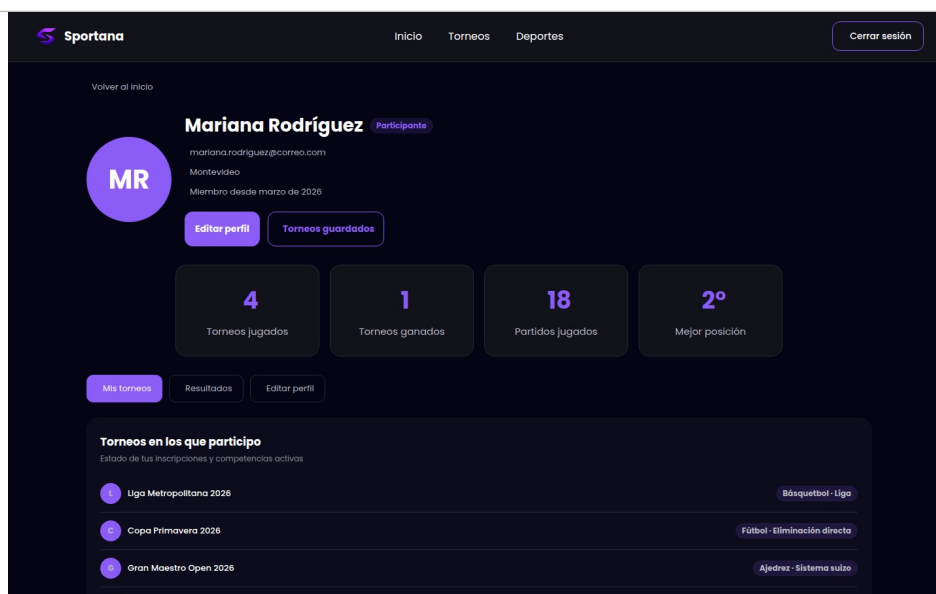


Figura 6. Perfil de usuario. Cabecera de perfil con avatar, datos del usuario y acciones disponibles.

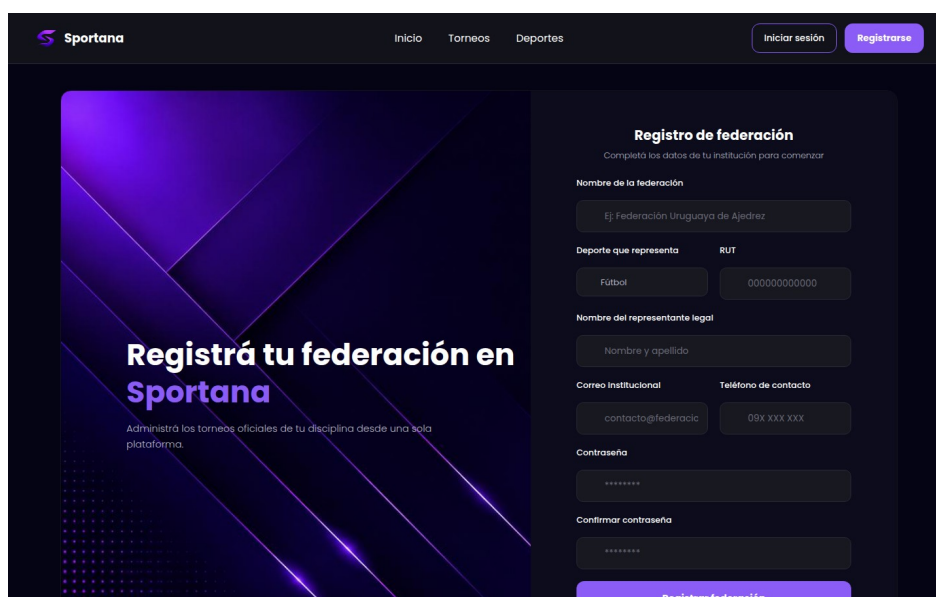


Figura 7. Registro de federación. Formulario de registro institucional, diferenciado del registro de un participante individual.

3.6 Accesibilidad y buenas prácticas

Se relevaron algunas prácticas de accesibilidad aplicadas de forma consistente en el maquetado. Las 64 imágenes presentes en el sitio incluyen el atributo alt (Mozilla Developer Network, 2026) en la totalidad de los casos; todas las páginas declaran la etiqueta meta viewport necesaria para que el navegador respete el ancho real del dispositivo en

lugar de simular un ancho de escritorio; y se utilizan etiquetas semánticas en lugar de div genéricos para estructurar el contenido: se registran 12 apariciones de header y de footer, 30 de section y 27 de article a lo largo de las doce páginas.

Queda como punto de mejora para una próxima entrega la incorporación sistemática de la etiqueta nav para las barras de navegación —presente actualmente solo en torneos.html— y de la etiqueta main para delimitar el contenido principal de cada página, ambas contempladas en la especificación de HTML5 pero todavía no aplicadas de forma uniforme en el sitio.

3.7 Elementos HTML y propiedades CSS utilizados

Se detalla a continuación el listado completo de etiquetas HTML y de propiedades CSS presentes en el código del proyecto, relevado directamente sobre las doce páginas y las ocho hojas de estilo, junto con la aplicación concreta que cada una cumple en Sportana.

3.7.1 Etiquetas HTML

Etiqueta	Para qué se utiliza en el proyecto
div	Contenedor genérico sin significado semántico, usado para agrupar y dar estructura o estilo a bloques de contenido.
span	Contenedor de texto en línea, usado para aplicar un estilo puntual (por ejemplo un color) sin alterar el flujo del párrafo.
p	Párrafo de texto: subtítulos, descripciones de torneos y deportes, textos de ayuda.
i	Ícono de Font Awesome, insertado mediante clases (por ejemplo fa-solid fa-magnifying-glass).
a	Enlace de navegación entre páginas o hacia una acción puntual (por ejemplo “Ver detalles” o “Ver todos”).
li	Elemento de una lista, usado dentro de ul para ítems de navegación o listados de reglamento.
h3	Subtítulo de tercer nivel, usado dentro de tarjetas y bloques de contenido (nombre de un deporte, título de un paso del formulario).
option	Opción dentro de un menú desplegable (select), por ejemplo cada deporte disponible en el filtro.
img	Imagen: portadas de torneos, ilustraciones de cada deporte,

Etiqueta	Para qué se utiliza en el proyecto
	avatar de perfil.
ul	Lista no numerada, usada para agrupar elementos de navegación o de listados.
label	Etiqueta de un campo de formulario, asociada a su input mediante el atributo for.
h2	Subtítulo de segundo nivel, usado como encabezado de subsecciones dentro de una página (por ejemplo “Datos generales” en el formulario de creación).
input	Campo de entrada de datos en los formularios: texto, fecha, número, checkbox.
button	Botón interactivo: filtrar, iniciar sesión, avanzar de paso en el formulario de creación.
link	Enlace a una hoja de estilos externa (Font Awesome o los propios CSS del proyecto), ubicado en el head de cada página.
section	Sección semántica de una página, por ejemplo el listado de torneos o el hero de portada.
article	Bloque de contenido autocontenido, usado en cada tarjeta de torneo o de deporte.
meta	Metadatos de la página: codificación de caracteres, viewport para el diseño responsive, descripción.
title	Título que se muestra en la pestaña del navegador.
html	Elemento raíz que envuelve todo el documento.
hr	Línea divisoria horizontal entre bloques de contenido.
header	Encabezado de la página, con el logo de Sportana y la navegación principal.
head	Contenedor de metadatos y enlaces a las hojas de estilo, no visible en la página.
h1	Título principal de cada página (por ejemplo “Explorar torneos”).
footer	Pie de página, común a las doce páginas del sitio.
body	Contenedor de todo el contenido visible de la página.
form	Formulario: inicio de sesión, registro, creación de torneo.
select	Menú desplegable: filtro de deporte, de módulo de competencia, de fecha.

Etiqueta	Para qué se utiliza en el proyecto
textarea	Campo de texto de varias líneas, usado para la descripción y el reglamento del torneo.
nav	Contenedor semántico de navegación, usado en torneos.html.

Tabla 3. Listado completo de etiquetas HTML utilizadas en el proyecto.

3.7.2 Propiedades CSS

Propiedad	Para qué se utiliza en el proyecto
color	Define el color del texto.
font-size	Define el tamaño de la tipografía.
display	Define el modelo de caja de un elemento: flex, inline-flex, none, etc.
background-color	Define el color de fondo de un elemento.
gap	Define el espacio entre los elementos hijos de un contenedor flex, sin necesidad de márgenes individuales.
flex-direction	Define si los elementos de un contenedor se disponen en fila o en columna.
align-items	Alinea los elementos hijos en el eje transversal de un contenedor flex.
padding	Define el espacio interno entre el borde de un elemento y su contenido.
font-weight	Define el grosor de la tipografía (normal, negrita, u otros valores intermedios).
width	Define el ancho de un elemento.
margin	Define el espacio externo alrededor de un elemento.
transition	Define una animación suave entre dos estados de un elemento, por ejemplo al pasar el mouse sobre una tarjeta.
border-radius	Redondea las esquinas de un elemento (tarjetas, botones, campos de formulario).
justify-content	Alinea los elementos hijos en el eje principal de un contenedor flex.
border	Define el borde de un elemento: grosor, estilo y color.
text-align	Alinea el texto dentro de su contenedor: izquierda, centro o derecha.

Propiedad	Para qué se utiliza en el proyecto
border-color	Define el color del borde de un elemento.
margin-bottom	Define el espacio externo debajo de un elemento.
margin-top	Define el espacio externo encima de un elemento.
height	Define el alto de un elemento.
line-height	Define el alto de línea del texto, afectando el espacio entre renglones.
position	Define el modo de posicionamiento de un elemento: relative, absolute, etc.
cursor	Define el ícono del cursor al pasar sobre un elemento, por ejemplo una mano sobre los botones.
text-decoration	Define el subrayado u otros adornos del texto; se usa principalmente para quitarle el subrayado por defecto a los enlaces.
flex	Define cómo crece o se achica un elemento dentro de un contenedor flex respecto de sus hermanos.
border-top	Define el borde superior de un elemento, usado como línea divisoria en encabezados.
transition-timing-function	Define la curva de aceleración de una transición (por ejemplo, un efecto más marcado al principio o al final).
max-width	Define el ancho máximo que puede alcanzar un elemento, aunque su contenedor sea más ancho.
transform	Aplica transformaciones visuales a un elemento, como una rotación o un cambio de escala al interactuar.
min-width	Define el ancho mínimo que debe conservar un elemento.
left	Define la distancia desde el borde izquierdo en un elemento posicionado.
white-space	Define cómo se maneja el espacio en blanco y el salto de línea dentro de un texto.
overflow	Define qué sucede cuando el contenido excede el tamaño de su contenedor (ocultarlo, recortarlo o mostrar scroll).
padding-left	Define el espacio interno del lado izquierdo de un elemento.
padding-right	Define el espacio interno del lado derecho de un elemento.
top	Define la distancia desde el borde superior en un elemento posicionado.

Propiedad	Para qué se utiliza en el proyecto
flex-wrap	Permite que los elementos de un contenedor flex continúen en una nueva línea cuando no entran en el ancho disponible.
pointer-events	Define si un elemento puede recibir interacciones del mouse; se usa para que un ícono decorativo no interfiera con un clic.
font-family	Define la tipografía utilizada (Poppins, con alternativas del sistema).
object-fit	Define cómo se ajusta una imagen dentro de su contenedor sin deformarse.
right	Define la distancia desde el borde derecho en un elemento posicionado.
padding-bottom	Define el espacio interno inferior de un elemento.
padding-top	Define el espacio interno superior de un elemento.
overflow-x	Define qué sucede cuando el contenido excede el ancho de su contenedor.
border-bottom	Define el borde inferior de un elemento.
background	Propiedad abreviada para definir el fondo de un elemento.
bottom	Define la distancia desde el borde inferior en un elemento posicionado.
min-height	Define el alto mínimo que debe conservar un elemento.
appearance	Elimina el estilo por defecto que el navegador aplica a un elemento, usado en los menús desplegables personalizados.
align-self	Sobrescribe, para un elemento puntual, la alineación definida por align-items en su contenedor.
background-image	Define una imagen de fondo, como la del hero de la página de inicio.
background-size	Define cómo se escala una imagen de fondo dentro de su contenedor (por ejemplo, cubriendo todo el espacio disponible).
box-sizing	Define si el padding y el borde de un elemento se incluyen dentro de su ancho total declarado.
list-style	Define o quita las viñetas por defecto de una lista.
resize	Permite o impide que la persona usuaria cambie el tamaño de un campo de texto.
margin-right	Define el espacio externo a la derecha de un elemento.

Propiedad	Para qué se utiliza en el proyecto
margin-left	Define el espacio externo a la izquierda de un elemento.

Tabla 4. Listado completo de propiedades CSS utilizadas en el proyecto.

3.8 Conclusión general

El presente documento describió el maquetado completo del sitio Sportana desarrollado para el proyecto SGDM, aplicando la metodología Mobile First mediante Flexbox como modelo de layout único, con un sistema de diseño visual y una organización de archivos definidos. Se relevaron las doce páginas construidas, sus funcionalidades de interfaz y las decisiones de accesibilidad aplicadas hasta el momento. Estos elementos constituyen la base sobre la cual se integrará, en las siguientes entregas, la lógica de servidor en PHP, el modelo de base de datos y la comunicación asincrónica descriptos en la letra de proyecto.

4. Bibliografía

Font Awesome. (2026). *Font Awesome 6 — Icons*. <https://fontawesome.com>

Mozilla Developer Network. (2026a). *CSS flexible box layout*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_flexible_box_layout

Mozilla Developer Network. (2026b). *: The Image Embed element*. MDN Web Docs. <https://developer.mozilla.org/en-US/docs/Web/HTML/Element/img>

Mozilla Developer Network. (2026c). *Using media queries*. MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_media_queries/Using_media_queries

Wroblewski, L. (2011). *Mobile First. A Book Apart*. <https://mobile-first.abookapart.com/>

PARTE II

Segunda Entrega

Modelo relacional, DCL, integración con PHP en POO e implementación con Apache

La Parte I (páginas anteriores) corresponde a la Primera Entrega — maquetado del sitio. Índice, numeración de página y bibliografía de esta Parte II son propios, independientes de la Parte I.

1. Índice

2. Vocabulario y Simbología.....	3
3. Desarrollo del tema.....	3
3.1 Introducción.....	3
3.1.1 Esquema de pantallas.....	4
3.2 Modelo Entidad-Relación (MER).....	4
3.3 Modelo Relacional normalizado.....	5
3.3.1 Justificación de la normalización.....	7
3.4 DCL implementado.....	8
3.5 Configuración de usuarios de base de datos con restricciones.....	9
3.6 Implementación de los modelos.....	10
3.7 Integración con PHP en Programación Orientada a Objetos.....	10
3.7.1 El asistente de crear torneo: tres correcciones necesarias.....	11
3.7.2 Comunicación asíncrona con JavaScript.....	11
3.7.3 Inscripción a un torneo: quién la hace, y detalle.html pasa de estática a dinámica.....	11
3.8 Implementación con Apache.....	12
3.9 Evidencia de pruebas realizadas.....	12
3.9.1 Pruebas de la clase Usuario (ejecutadas por línea de comandos).....	12
3.9.2 Prueba de extremo a extremo contra Apache real.....	13
3.9.3 Prueba de extremo a extremo del flujo de federación.....	13
3.9.4 Prueba de extremo a extremo de la creación de torneos.....	14
3.9.5 Control de acceso en la creación de torneos.....	14
3.9.6 Datos de prueba cargados para la verificación integral.....	14
3.9.7 Inscripción de un participante, hecha por el organizador.....	15
3.9.8 Reglas específicas de un torneo (configuracion_torneo).....	15

3.9.9 Auditoría cruzada contra el diagrama de clases real de Ingeniería de Software.....	15
3.10 Relación con Ciberseguridad.....	16
3.11 Conclusión.....	17
4. Bibliografía.....	17
Hoja Testigo.....	18

2. Vocabulario y Simbología

A continuación se definen los principales términos técnicos utilizados a lo largo de este documento.

- **DDL:** Lenguaje de Definición de Datos: subconjunto de SQL que crea y modifica la estructura de una base de datos (CREATE TABLE, ALTER TABLE).
- **DML:** Lenguaje de Manipulación de Datos: subconjunto de SQL que opera sobre los datos (SELECT, INSERT, UPDATE, DELETE).
- **DCL:** Lenguaje de Control de Datos: subconjunto de SQL que administra permisos de acceso (GRANT, REVOKE).
- **Normalización:** proceso de organizar una base de datos relacional para reducir la redundancia y evitar anomalías de actualización, inserción y borrado.
- **Tercera Forma Normal (3FN):** nivel de normalización en el que cada atributo no clave depende únicamente de la clave primaria completa, y no de otros atributos no clave.
- **POO:** Programación Orientada a Objetos: paradigma que organiza el código en clases con atributos y métodos propios.
- **mysqli:** extensión de PHP que permite ejecutar consultas SQL contra bases de datos MySQL/MariaDB, con soporte para consultas preparadas.
- **Consulta preparada:** técnica que separa la consulta SQL de los datos ingresados por el usuario, evitando inyección SQL al enviar los valores por separado en vez de concatenarlos al texto de la consulta.
- **password_hash():** función nativa de PHP que transforma una contraseña en un hash irreversible mediante el algoritmo bcrypt, para no almacenar contraseñas en texto plano.
- **Principio de mínimo privilegio:** principio de seguridad según el cual cada usuario o proceso debe contar únicamente con los permisos estrictamente necesarios para cumplir su función.
- **Apache:** servidor HTTP de código abierto utilizado para servir la aplicación web y ejecutar los scripts PHP.

3. Desarrollo del tema

3.1 Introducción

El presente documento corresponde a la segunda entrega de la unidad curricular Programación Full Stack, en el marco del proyecto Sistema de Gestión Deportiva Modular (SGDM), desarrollado por la empresa Zenko bajo el nombre comercial Sportana. Mientras que la primera entrega correspondió al maquetado de la interfaz, esta segunda entrega desarrolla la capa de datos y el backend: el modelo relacional normalizado, el control de acceso a la base de datos (DCL), la implementación de PHP en programación orientada a objetos, y el despliegue con Apache.

Todo el contenido de este documento corresponde a código que fue efectivamente escrito y ejecutado —no solo diseñado— contra una base de datos MariaDB y un servidor Apache reales, con evidencia de las pruebas realizadas en la sección 3.9. Es importante aclarar que estas pruebas se realizaron en un entorno de prueba, no en el servidor AlmaLinux del proyecto (documentado en Sistemas Operativos): el código queda listo para desplegar, pero el despliegue en el servidor real del equipo es un paso pendiente, con instrucciones específicas para AlmaLinux incluidas junto al código entregado.

3.1.1 Esquema de pantallas

El sitio Sportana, construido en la primera entrega, consta de doce páginas HTML. La siguiente tabla las agrupa por función e indica cuáles quedaron conectadas a un endpoint real en esta segunda entrega:

Página	Función	Backend en esta entrega
index.html	Portada, torneos destacados	No — contenido estático
torneos.html	Explorar y filtrar torneos	No — contenido estático
deportes.html	Listado de deportes disponibles	No — contenido estático
detalle.html	Detalle de un torneo	Sí — obtener_torneo.php (lectura) y procesar_inscripcion.php
registro.html	Alta de cuenta de participante	Sí — procesar_registro.php
login.html	Inicio de sesión (participante, organizador, admin)	Sí — procesar_login.php
registro-federacion.html	Alta de cuenta de federación	Sí — procesar_registro_federacion.php
login-federacion.html	Inicio de sesión de federación	Sí — procesar_login_federacion.php
crear.html	Crear torneo, paso 1 de 3 (datos y módulo)	Parcial — guarda en sessionStorage
crear2.html	Crear torneo, paso 2 de 3 (detalles y premios)	Parcial — guarda en sessionStorage
crear3.html	Crear torneo, paso 3 de 3 (participantes)	Sí — procesar_crear_torneo.php, combina los 3 pasos
perfil.html	Perfil del usuario autenticado	No — pendiente para próxima entrega

Tabla 1. Esquema de pantallas del sitio Sportana, con su estado de conexión al backend en esta entrega.

3.2 Modelo Entidad-Relación (MER)

Antes de pasar al modelo relacional, se modeló el sistema en su forma conceptual: el Modelo Entidad-Relación, en notación de Chen. Las entidades son rectángulos con

solo su nombre, los atributos son óvalos conectados a su entidad (con un círculo lleno para el atributo identificador), y cada relación entre dos entidades es un rombo propio, con la cardinalidad de participación de cada lado escrita como (mínimo, máximo) —por ejemplo, (0,n) del lado de rol en la relación tiene significa que un rol puede estar en cero o muchos usuarios.

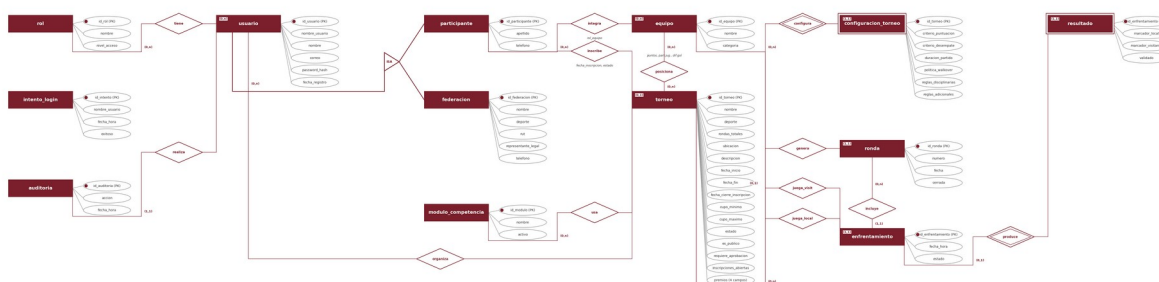


Figura 1. Modelo Entidad-Relación (MER) de Sportana en notación de Chen: 13 entidades, sus atributos, y las 13 relaciones (rombos) que las conectan, cada una con su cardinalidad de participación. Diagrama de alta resolución — hacé zoom en el PDF para ver el detalle completo.

Tres relaciones muchos-a-muchos tienen atributos propios y se representan como rombos, no como entidades: integra (equipo-participante, con el atributo rol_equipo), inscribe (torneo-participante, con fecha_inscripcion y estado) y posiciona (torneo-equipo, con puntos, partidos_jugados y diferencia_gol). enfrentamiento se relaciona con equipo dos veces —como local y como visitante—, mediante dos rombos separados (juega_local y juega_visit), ya que un único rombo no puede representar correctamente dos roles distintos de la misma entidad dentro de la misma relación.

El modelo no tiene ningún caso de agregación (una relación que participe, a su vez, de otra relación): ninguna de las 13 relaciones de este MER necesita conectarse con otra relación en lugar de con una entidad. Se deja constancia de esto en vez de forzar un caso que no corresponde al diseño real.

Se usan tres elementos adicionales de notación formal: la generalización (triángulo ISA) entre usuario y sus especializaciones, participante y federacion; el doble borde de entidad débil en configuracion_torneo y resultado, que no tienen identidad propia sin su entidad fuerte; y el rombo de doble línea en esas mismas dos relaciones, marcando que la participación de la entidad débil es total.

3.3 Modelo Relacional normalizado

El modelo relacional se derivó del diagrama de clases presentado en la segunda entrega de Ingeniería de Software, llevando cada clase UML a una tabla en tercera forma normal (3FN). Se agregó además una tabla para federaciones deportivas (no modelada como clase separada en el diagrama original, pero necesaria dado que el registro de federación captura datos distintos a los de un participante) y una tabla para registrar intentos de inicio de sesión, requerida por la política de bloqueo de cuentas definida en Ciberseguridad.

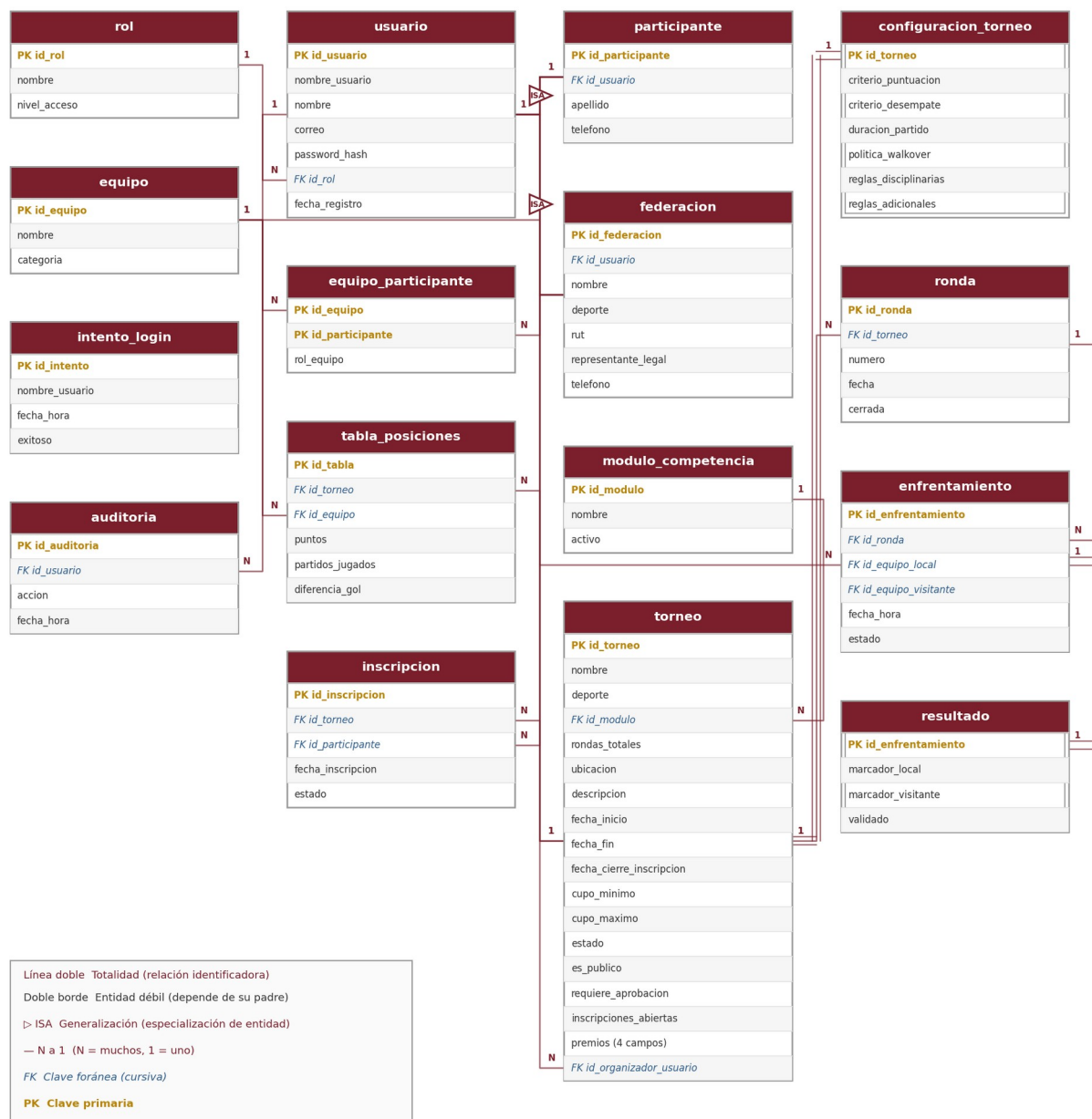


Figura 2. Modelo Relacional de Sportana: las 16 tablas ya normalizadas, con sus columnas, claves primarias y foráneas, y la cardinalidad de cada relación. Es la implementación lógica del MER de la Figura 1: los rombos integra, inscribe y posiciona pasan a ser las tablas equipo_participante, inscripcion y tabla_posiciones respectivamente.

La Figura 2 conserva los mismos tres elementos formales del MER — generalización, entidad débil y totalidad, explicados en la sección 3.2— aplicados ahora sobre las tablas reales: el triángulo ISA entre usuario y sus especializaciones, el doble borde en configuracion_torneo y resultado, y la línea doble en sus relaciones identificadoras con torneo y enfrentamiento.

Tabla	Propósito
rol	Catálogo de roles del sistema (participante, organizador, administrador).
usuario	Cuenta base de cualquier persona o entidad que accede al sistema: nombre, credenciales y contraseña ya hasheada.
participante	Apellido y teléfono de un usuario cuando se registra como participante (relación 1 a 0..1 con usuario); el nombre se hereda de usuario.
federacion	Datos adicionales de un usuario cuando se registra como federación deportiva.
equipo	Equipos que pueden inscribirse a torneos.
equipo_participante	Tabla puente entre equipo y participante (muchos a muchos), con el rol de cada integrante dentro de su equipo.
modulo_competencia	Catálogo de formatos de competencia: liga, eliminación directa, sistema suizo.
torneo	Datos generales de cada torneo, incluyendo visibilidad pública, aprobación de inscripción y premios.
configuracion_torneo	Reglas específicas de un torneo (puntuación, desempate, duración, walkover, disciplina), entidad débil que depende de torneo (relación 1 a 1).
ronda	Rondas de un torneo.
enfrentamiento	Partidos entre equipos dentro de una ronda.
resultado	Resultado de un enfrentamiento (relación 1 a 0..1, ya que puede no estar cargado aún).
tabla_posiciones	Puntos, partidos jugados y diferencia de gol de cada equipo en un torneo.
auditoria	Registro de acciones relevantes realizadas por los usuarios.
intento_login	Registro de cada intento de inicio de sesión, exitoso o no, para aplicar el bloqueo temporal tras 5 fallos.
inscripcion	Inscripción individual de un participante a un torneo, con su estado (pendiente/aprobada) según si el torneo requiere aprobación.

Tabla 2. Las 16 tablas del modelo relacional normalizado, con su propósito.

El script completo (sql/01_modelo_relacional.sql) fue ejecutado contra una instancia real de MariaDB 10.11, creando las 16 tablas sin errores, con sus claves primarias, foráneas y restricciones de unicidad correspondientes.

3.3.1 Justificación de la normalización

El modelo cumple 1FN, 2FN y 3FN por las siguientes razones concretas, verificables contra el esquema real:

- **Primera forma normal (1FN):** todos los atributos son atómicos; no hay campos que almacenen listas ni valores compuestos. Por ejemplo, los cuatro premios de un torneo son cuatro columnas independientes (premio_primero, premio_segundo, premio_tercero, premio_mvp) en lugar de un único campo con los cuatro valores separados por comas.
- **Segunda forma normal (2FN):** la única clave primaria compuesta del modelo es equipo_participante (id_equipo, id_participante). Su único atributo no clave, rol_equipo, depende de ambas columnas de la clave en conjunto —el rol de una persona solo tiene sentido dentro de un equipo específico—, no de una sola de ellas. No hay ninguna otra tabla con clave compuesta, por lo que no hay más dependencias parciales que evaluar.
- **Tercera forma normal (3FN):** se evitaron dependencias transitivas separando en tablas propias todo atributo que no dependiera directamente de la clave primaria de la tabla que lo contenía. Tres decisiones concretas de diseño responden a esto:
- **rol como catálogo aparte:** nombre y nivel_acceso no se repiten en cada fila de usuario; se referencian mediante id_rol. Guardarlos directamente en usuario haría que nivel_acceso dependiera de nombre del rol, y no de id_usuario.
- **participante y federacion separadas de usuario:** apellido, telefono o rut no son aplicables a todas las cuentas por igual —una federación no tiene apellido, un participante no tiene rut—, y mezclarlos en una única tabla usuario dejaría columnas sistemáticamente vacías según el tipo de cuenta. Separarlos en participante y federacion, cada una con su propia clave primaria y una referencia a usuario, evita esa mezcla de conceptos distintos en una misma tabla.
- **modulo_competencia como catálogo aparte:** liga, eliminación directa y sistema suizo no se escriben como texto libre en cada torneo; se referencian mediante id_modulo. Esto evita inconsistencias de escritura entre torneos y permite agregar un nuevo módulo sin tocar la tabla torneo.

Hay una excepción deliberada, no un error: tabla_posiciones almacena puntos, partidos_jugados y diferencia_gol de forma directa, aunque estos valores podrían calcularse a partir de los datos ya existentes en enfrentamiento y resultado. Se optó por esta redundancia controlada —una práctica común y aceptada, no una violación de 3FN por descuido— porque calcular la tabla de posiciones de un torneo en cada consulta implicaría recorrer todos sus enfrentamientos y resultados cada vez que alguien la mire, en lugar de leer una fila ya calculada.

3.4 DCL implementado

El informe de Ciberseguridad (sección 3) ya define cuatro roles con permisos distintos —Administrador General, Organizador, Usuario y Usuario Público— bajo el principio de mínimo privilegio. El DCL implementa esa misma separación a nivel de base de datos: un usuario de MariaDB por cada rol del sistema, además del usuario de autenticación que usa la aplicación para validar credenciales antes de conocer el rol.

Usuario de BD	Rol del sistema	Permisos otorgados
sportana_app	Autenticación	SELECT/INSERT/UPDATE/DELETE en toda la

Usuario de BD	Rol del sistema	Permisos otorgados
	(login/registro)	base — necesita ubicar cualquier cuenta por sus credenciales antes de saber su rol.
sportana_publico	Usuario Público	SELECT únicamente, sobre torneo, ronda, enfrentamiento, resultado, tabla_posiciones, equipo, modulo_competencia, inscripcion y vista_organizador. Sin acceso directo a usuario ni federacion (la vista solo expone el nombre del organizador, no columnas sensibles).
sportana_participante	Usuario (participante)	SELECT sobre las mismas tablas públicas; además INSERT en equipo_participante (inscribirse) y UPDATE en participante (su propio perfil).
sportana_organizador	Organizador	SELECT/INSERT/UPDATE sobre torneo, configuracion_torneo, ronda, enfrentamiento, resultado, equipo, equipo_participante y tabla_posiciones. Sin acceso a usuario, rol, participante, federacion ni auditoria.
sportana_admin	Administrador General	SELECT/INSERT/UPDATE/DELETE en toda la base, sin DROP/ALTER/GRANT OPTION: administra datos, no la estructura de la base.

Tabla 3. Los cinco usuarios de base de datos del DCL, con el rol de Ciberseguridad que cada uno implementa.

Cada restricción se verificó conectándose realmente como cada usuario y probando operaciones que debían fallar, no solo revisando el GRANT escrito:

```
$ mariadb -u sportana_publico -p sportana -e "INSERT INTO torneo (...) VALUES (...);"
ERROR 1142 (42000): INSERT command denied to user 'sportana_publico'@'localhost' for table `torneo`
```

```
$ mariadb -u sportana_organizador -p sportana -e "SELECT * FROM usuario;"
ERROR 1142 (42000): SELECT command denied to user 'sportana_organizador'@'localhost' for table `usuario`
```

```
$ mariadb -u sportana_organizador -p sportana -e "DELETE FROM torneo WHERE ...;"
ERROR 1142 (42000): DELETE command denied to user 'sportana_organizador'@'localhost' for table `torneo`
```

```
$ mariadb -u sportana_participante -p sportana -e "INSERT INTO torneo (...) VALUES (...);"
ERROR 1142 (42000): INSERT command denied to user 'sportana_participante'@'localhost' for table `torneo`
```

Intentos reales de operaciones fuera de los permisos de cada rol, todos rechazados por MariaDB.

De estos cinco usuarios, la aplicación PHP usa hoy activamente sportana_app para las operaciones de autenticación (sección 3.7) y para resolver el correo de un participante al inscribirlo, sportana_organizador para la creación de torneos y para inscribir participantes (secciones 3.7.1 y 3.7.3), y sportana_publico para la lectura

pública de un torneo, todo verificado con `CURRENT_USER()` en las secciones 3.9.5 y 3.9.7. `sportana_participante` quedó provisionado pero sin uso activo: la letra de proyecto (5.3) no le da al participante ninguna acción de escritura sobre torneos, solo consulta. `sportana_admin` tampoco tiene uso activo todavía, para cuando se implemente, en la tercera entrega, la funcionalidad de administración general.

3.5 Configuración de usuarios de base de datos con restricciones

La aplicación PHP nunca se conecta a la base de datos como root. La clase `Conexion` (sección 3.7) lee las credenciales del usuario `sportana_app` desde variables de entorno, nunca hardcodeadas en el código fuente, de modo que la contraseña real no queda expuesta en el repositorio del proyecto.

Este diseño aplica el principio de mínimo privilegio: si existiera una falla de seguridad en el código PHP, el atacante quedaría limitado a los permisos de `sportana_app` —leer y modificar datos dentro de la base `sportana`— sin poder borrar tablas, alterar la estructura ni acceder a otras bases del servidor.

3.6 Implementación de los modelos

Además de la estructura, se cargaron los datos base necesarios para que el sistema sea funcional desde el primer momento: los tres roles (participante, organizador, administrador) y los tres módulos de competencia (liga, eliminación directa, sistema suizo), ambos definidos en la letra de proyecto y en el ESRE de Ingeniería de Software.

3.7 Integración con PHP en Programación Orientada a Objetos

El backend se implementó mediante cinco clases PHP, cubriendo cuentas de participante, cuentas de federación deportiva, la creación de torneos y la inscripción a ellos:

- **Conexion:** envuelve la conexión `mysql` en una única instancia reutilizable (patrón `Singleton`), leyendo `host`, `usuario`, `contraseña`, `base` y `puerto` desde variables de entorno.
- **Usuario:** implementa el registro (registrar) y el inicio de sesión (login) de participantes, con validación de datos, hash de contraseñas mediante `bcrypt`, consultas preparadas para evitar inyección SQL, y el bloqueo temporal de cuentas tras intentos fallidos.
- **Federacion:** implementa el mismo patrón para cuentas de federación deportiva, con la diferencia de que el inicio de sesión acepta tanto el RUT como el correo institucional como identificador, tal como lo pide el campo único de `login-federacion.html`.
- **Torneo:** implementa la creación de torneos (crear) a partir de los tres pasos del asistente, validando campos obligatorios, el módulo de competencia elegido y la coherencia entre fecha de inicio y fin. También crea el registro de

configuracion_torneo asociado, con las reglas específicas del torneo (antes de esta corrección, esa tabla existía en el esquema pero ningún código la llenaba).

- **Inscripcion:** implementa la inscripción individual de un participante a un torneo (crear), respetando si las inscripciones están abiertas, si el plazo de inscripción ya cerró, y el cupo máximo del torneo.

El registro se ejecuta dentro de una transacción (begin_transaction / commit / rollback), de modo que si falla la inserción en la tabla participante o federación después de haber creado el usuario, ambas operaciones se revierten y no queda una cuenta a medio crear.

Las contraseñas se validan contra la política definida en Ciberseguridad —mínimo 8 caracteres, mayúscula, minúscula, número y carácter especial— antes de intentar cualquier operación sobre la base de datos, y se almacenan siempre mediante password_hash() con el algoritmo bcrypt, nunca en texto plano.

3.7.1 El asistente de crear torneo: tres correcciones necesarias

crear.html, crear2.html y crear3.html reparten la creación de un torneo en tres páginas separadas. Al conectarlas con el backend surgieron tres problemas reales —dos del maquetado original, uno de la primera versión de este mismo backend—, que hubo que corregir para que la funcionalidad fuera genuina y, además, segura:

- **Pérdida de datos entre pasos:** los botones “Siguiente” entre pasos eran enlaces comunes (<a href>): al navegar de una página a la siguiente, los datos ya cargados se perdían por completo. Se agregó JavaScript que guarda los datos de cada paso en sessionStorage al avanzar, y en el paso 3 se combinan los tres pasos y se envían juntos al backend.
- **Interruptores sin valor real:** los tres interruptores del paso 3 (“Torneo público”, “Requiere aprobación de inscripción”, “Inscripciones abiertas”) eran puramente visuales, sin ningún <input> asociado: no había forma de que su estado viajara con el formulario. Se agregó un checkbox oculto detrás de cada uno, sincronizado por JavaScript con el interruptor visual, sin cambiar su apariencia.
- **Falta de control de acceso:** la primera versión de procesar_crear_torneo.php no verificaba que hubiera una sesión iniciada: cualquiera podía crear un torneo sin registrarse ni loguearse, y quedaba sin registrar quién lo había creado. Se agregó la verificación de sesión y de rol (organizador o administrador, según Ciberseguridad secciones 3.1 y 3.2), y la conexión a la base de datos pasó de sportana_app a sportana_organizador —el usuario de BD que corresponde a esta operación según el DCL (sección 3.4)—, que hasta entonces estaba provisionado pero sin usarse.

Los dos primeros problemas ya existían en el maquetado heredado de la primera entrega; el tercero se detectó al revisar con ojo crítico el propio backend construido en esta entrega. Ninguno es un error introducido a propósito: son las condiciones que había que corregir para que “Integración con PHP” describa una integración que efectivamente funciona y respeta los permisos de cada rol.

3.7.2 Comunicación asincrónica con JavaScript

La letra de proyecto especifica JavaScript como parte del stack de frontend (secciones 4.1 y 6) y, en particular, la competencia CET4 pide explícitamente “implementar operaciones CRUD y comunicación asincrónica” integrando front-end y back-end. En base a esto, los cuatro formularios de autenticación (registro.html, login.html, registro-federacion.html, login-federacion.html) y el paso final del asistente de crear torneo (crear3.html) se conectaron a sus endpoints mediante fetch(), en lugar de un envío tradicional de formulario con recarga de página.

Cada formulario intercepta el evento submit, envía los datos por POST de forma asincrónica, y muestra el resultado —éxito o el mensaje de error específico devuelto por el backend— directamente en la página, sin navegar a una URL distinta. Esto evita que el usuario vea la respuesta JSON cruda del servidor, y deja la validación de backend (Ciberseguridad, sección 5) efectivamente visible en la interfaz.

3.7.3 Inscripción a un torneo: quién la hace, y detalle.html pasa de estática a dinámica

detalle.html tenía un botón “Inscribirme” (`<button type="button">`) sin ningún dato asociado: no indicaba a qué torneo correspondía, y toda la página mostraba valores fijos de ejemplo. La primera versión de esta funcionalidad asumió inscripción individual, con el participante anotándose a sí mismo. Al revisar la letra de proyecto (sección 5.2 y 5.3) quedó claro que ese supuesto era incorrecto: es el organizador quien “inscribe participantes” a su torneo; el participante, en su propia lista de permisos (5.3), no tiene esa acción.

Se corrigió el flujo completo para reflejar esto. La tabla inscripcion, que vincula participante y torneo directamente, se mantiene igual; lo que cambió es quién puede insertar en ella. `procesar_inscripcion.php` ahora exige sesión de organizador o administrador —no de participante—, y recibe el correo del participante a inscribir en lugar de tomarlo de la sesión propia. El botón se renombró a “Inscribir participante”, y al hacer clic pide ese correo.

Esto obligó a resolver una tensión real de permisos: para inscribir por correo, el sistema necesita poder buscar en la tabla participante, pero `sportana_organizador` no tiene acceso a ella —Ciberseguridad (3.2) es explícito en que el organizador no administra usuarios—. La búsqueda del correo se resuelve con la conexión de autenticación (`sportana_app`), y recién la inscripción en sí queda escrita por `sportana_organizador`, verificado con `CURRENT_USER()` en la sección 3.9.7.

detalle.html toma el id del torneo de la URL (`?id=`) y pide sus datos reales a `obtener_torneo.php` (lectura pública, sin necesidad de sesión), completando título, fechas, ubicación, inscriptos y cupos disponibles por JavaScript.

3.8 Implementación con Apache

Se conectaron los formularios reales de registro.html y login.html —construidos en la primera entrega— a dos endpoints PHP (`procesar_registro.php` y `procesar_login.php`) que reciben los datos por POST y devuelven una respuesta en formato JSON. registro.html se modificó únicamente para agregar los atributos `action` y `method`, sin

alterar ningún otro elemento. login.html requirió además un segundo ajuste: el campo pedía originalmente correo electrónico, y se cambió a nombre de usuario para que coincida con RF-14 de Ingeniería de Software, que especifica ese campo como identificador de inicio de sesión.

Las variables de entorno de conexión a la base de datos se configuran a nivel de Apache mediante un archivo .htaccess con directivas SetEnv, lo que evita tener que modificar el código PHP entre el entorno de prueba y el servidor de producción: alcanza con cambiar los valores del .htaccess.

Este flujo completo —formulario HTML servido por Apache, envío por POST, procesamiento en PHP, escritura y lectura en MariaDB— fue ejecutado de punta a punta contra un servidor Apache real, según se detalla en la sección siguiente.

3.9 Evidencia de pruebas realizadas

Se construyó una batería de pruebas (test_usuario.php) que ejercita el flujo completo contra la base de datos real, y se probó además el mismo flujo mediante peticiones HTTP reales contra Apache, simulando exactamente lo que hace un navegador al enviar un formulario.

3.9.1 Pruebas de la clase Usuario (ejecutadas por línea de comandos)

```
=== Test 1: registrar un participante nuevo ===
OK    - Usuario registrado con id=1

=== Test 2: login con contraseña correcta (usando nombre de usuario) ===
OK    - Login exitoso
OK    - Rol asignado correctamente: participante

=== Test 3: login con contraseña incorrecta ===
OK    - Login rechazado correctamente con contraseña mala

=== Test 5: no permite registrar el mismo usuario dos veces ===
OK    - Rechazado correctamente: El nombre de usuario o el correo ya están
registrados.

=== Test 6b: política de contraseñas – rechaza sin mayúscula/número/especial ===
OK    - Rechazada 'sinmayuscula1!': La contraseña debe incluir al menos una mayúscula.
OK    - Rechazada 'SinNumeroEspecial': La contraseña debe incluir al menos un número.

=== Test 7: verificar que la contraseña se guardó hasheada ===
OK    - El hash guardado no es la contraseña en texto plano
OK    - El hash tiene formato bcrypt real: $2y$10$wha...

=== Test 8: bloqueo tras 5 intentos fallidos ===
OK    - La cuenta terminó bloqueada tras varios intentos fallidos
OK    - Sigue bloqueada aunque la contraseña sea correcta
```

✓ TODOS LOS TESTS PASARON

Salida real de test_usuario.php, ejecutado contra una base de datos MariaDB real (extracto).

3.9.2 Prueba de extremo a extremo contra Apache real

```
$ curl -X POST http://localhost/procesar_registro.php \
-d
```

```
"nombre=Carla&apellido=Gomez&correo=carla@test.com&usuario=carla_g&contrasena=Password2026!&confirmar_contrasena=Password2026!"
{"ok":true,"id_usuario":3}
```

```
$ curl -X POST http://localhost/procesar_login.php \
-d "usuario=carla_g&contrasena=Password2026!"
{"ok":true,"rol":"participante","nombre_usuario":"carla_g"}
```

Peticiones HTTP POST reales contra el servidor Apache, con respuesta real de la aplicación.

3.9.3 Prueba de extremo a extremo del flujo de federación

Se repitió la misma prueba para el flujo de federación, verificando además que el login acepte indistintamente el RUT o el correo institucional como identificador, ambos válidos según login-federacion.html:

```
$ curl -X POST http://localhost/procesar_registro_federacion.php \
-d "nombre_federacion=Federacion de
Basquet&deporte_federacion=Basquetbol&rut=211234560012&representante=Juan
Diaz&correo_federacion=contacto@fedbasquet.uy&telefono=099123456&contrasena=Password2026!&confirmar_contrasena=Password2026!"
{"ok":true,"id_usuario":2}
```

```
$ curl -X POST http://localhost/procesar_login_federacion.php \
-d "identificador_federacion=211234560012&contrasena=Password2026!"
{"ok":true,"rol":"organizador","nombre":"Federacion de Basquet"}
```

```
$ curl -X POST http://localhost/procesar_login_federacion.php \
-d "identificador_federacion=contacto@fedbasquet.uy&contrasena=Password2026!"
{"ok":true,"rol":"organizador","nombre":"Federacion de Basquet"}
```

Registro de federación y login exitoso probando ambos identificadores válidos: RUT y correo institucional.

3.9.4 Prueba de extremo a extremo de la creación de torneos

Se probó el envío combinado de los tres pasos del asistente, un caso de error (módulo de competencia inválido) y que los tres formatos del paso 1 mapeen al módulo de competencia correcto en la base de datos:

```
$ curl -X POST http://localhost/procesar_crear_torneo.php \
-d "nombre=Liga Metropolitana 2026&organizador=Club
Nacional&deporte=Basquetbol&formato=liga&..."
{"ok":true,"id_torneo":1}
```

```
$ curl -X POST http://localhost/procesar_crear_torneo.php \
-d "nombre=Test&organizador=Test&deporte=Futbol&formato=invalido&..."
{"ok":false,"error":"El módulo de competencia seleccionado no es válido."}
```

```
-- Verificación en la base de datos de los 3 formatos enviados --
SELECT t.nombre, m.nombre AS modulo FROM torneo t JOIN modulo_competencia m ON
m.id_modulo=t.id_modulo;
Liga Metropolitana 2026    -> liga
Copa Relampago (eliminacion) -> eliminacion_directa
Ajedrez Suizo (suizo)      -> sistema_suizo
```

Creación real de torneo, rechazo de un módulo de competencia inválido, y verificación de que los tres formatos del formulario mapean al módulo correcto.

3.9.5 Control de acceso en la creación de torneos

Se probaron los tres escenarios posibles —sin sesión, con sesión pero rol sin permiso, y con sesión y rol correcto—, usando cookies reales para simular la sesión de un navegador:

```
$ curl -X POST http://localhost/procesar_crear_torneo.php -d "nombre=Test&..."
{"ok":false,"error":"Tenés que iniciar sesión para crear un torneo."} HTTP 401

-- Login como participante (Dana López), guardando la cookie de sesión --
$ curl -X POST http://localhost/procesar_crear_torneo.php -b cookies_participante.txt
-d "..."
{"ok":false,"error":"Tu cuenta no tiene permiso para crear torneos."} HTTP 403

-- Login como Federación de Ajedrez (rol organizador), guardando la cookie --
$ curl -X POST http://localhost/procesar_crear_torneo.php -b cookies_federacion.txt -d
"..."
{"ok":true,"id_torneo":1} HTTP 200

-- Verificación: el torneo quedó asociado a quien realmente lo creó --
SELECT id_organizador_usuario FROM torneo WHERE id_torneo=1; -> 2 (id de Federación
de Ajedrez)

-- Confirmación de que cada conexión usa su propio usuario de MariaDB --
SELECT CURRENT_USER() vía Conexion::obtener('app') -> sportana_app@localhost
SELECT CURRENT_USER() vía Conexion::obtener('organizador') ->
sportana_organizador@localhost
```

Los tres escenarios de control de acceso, y confirmación de que la conexión de organizador es real y distinta a la de autenticación.

3.9.6 Datos de prueba cargados para la verificación integral

Se cargó un conjunto de datos de prueba reutilizando las mismas cuentas y torneos ya usados en las secciones anteriores —no se inventaron personas ni torneos nuevos para este punto—, mediante `sql/03_datos_prueba.sql` (equipos y un torneo adicional) y `php/cargar_datos_prueba.php` (participantes, federación y torneos, cargados a través de las clases reales para que las contraseñas queden hasheadas igual que en producción, no como texto plano en un script SQL). Se verificó la carga con una consulta cruzada real:

usuarios	3	
participantes	2	(Carla Gómez, Bruno Reyes)
federaciones	1	(Federación de Básquet)
equipos	2	(Los Tigres, Águilas FC)
torneos	4	(Copa Verificación 2026, Liga Metropolitana 2026, Copa Relámpago, Ajedrez Suizo)

Conteo real por tabla tras cargar los datos de prueba, verificado con `SELECT COUNT(*)` contra la base de datos.

3.9.7 Inscripción de un participante, hecha por el organizador

Se probó que un participante ya no pueda inscribirse a sí mismo, que el organizador sí pueda inscribirlo por su correo, y que la inscripción quede escrita por `sportana_organizador` (no por `sportana_app` ni `sportana_participante`):

```
-- Hugo (participante) intenta inscribirse a sí mismo --
$ curl -b cookies_hugo.txt -X POST .../procesar_inscripcion.php \
```

```
-d "id_torneo=1&correo_participante=hugo@test.com"
{"ok":false,"error":"Solo organizadores o administradores pueden inscribir
participantes."} HTTP 403
```

```
-- Federación de Tenis (rol organizador) inscribe a Hugo por su correo --
$ curl -b cookies_tenis.txt -X POST ../procesar_inscripcion.php \
-d "id_torneo=1&correo_participante=hugo@test.com"
{"ok":true,"id_inscripcion":1,"estado":"aprobada"} HTTP 200
```

```
-- Confirmación: la inscripción quedó escrita por el usuario de BD correcto --
SELECT CURRENT_USER() vía Conexion::obtener('organizador') ->
sportana_organizador@localhost
```

Un participante ya no puede autoinscribirse; el organizador sí puede, y la escritura queda a nombre de sportana_organizador.

3.9.8 Reglas específicas de un torneo (configuracion_torneo)

Se verificó que las 6 reglas específicas del paso 2 del asistente se guarden realmente en configuracion_torneo, tabla que existía desde el esquema original pero que ningún código llenaba hasta esta corrección:

```
$ curl -b cookies_rugby.txt -X POST ../procesar_crear_torneo.php \
-d "nombre=Copa Rugby&...&criterio-puntuacion=4 puntos por victoria, 2 por empate\
&criterio-desempate=Diferencia de puntos&duracion-partido=2 tiempos de 40
minutos\
&politica-walkover=Derrota 0-30&reglas-disciplinarias=Tarjeta roja = 2
fechas..."
{"ok":true,"id_torneo":1}
```

```
SELECT * FROM configuracion_torneo WHERE id_torneo=1;
criterio_puntuacion: 4 puntos por victoria, 2 por empate
criterio_desempate: Diferencia de puntos
duracion_partido: 2 tiempos de 40 minutos
politica_walkover: Derrota 0-30
reglas_disciplinarias: Tarjeta roja = 2 fechas de suspensión
```

Las 6 reglas específicas del torneo, guardadas realmente en configuracion_torneo, no como un único texto libre.

3.9.9 Auditoría cruzada contra el diagrama de clases real de Ingeniería de Software

Se revisó el modelo completo contra la imagen real de la Figura 3 del informe de Ingeniería de Software (no solo contra su descripción en texto), clase por clase. La gran mayoría coincide: rol, auditoria, equipo_participante, ronda, enfrentamiento y resultado tienen los mismos atributos, relaciones y cardinalidades. Se encontraron y corrigieron dos diferencias estructurales reales.

- **nombre en el lugar equivocado de la jerarquía:** el diagrama de clases pone nombre en Usuario (heredado por sus especializaciones), y Participante solo agrega apellido y telefono. El esquema tenía nombre duplicado en participante, y usuario no tenía ningún campo de nombre propio. Se movió nombre a usuario y se sacó de participante.
- **organizador de texto libre, redundante con la FK:** torneo tenía un campo organizador de texto libre y, por separado, la FK id_organizador_usuario: la misma redundancia señalada en la auditoría de la sección anterior. Se eliminó

el campo de texto; el nombre del organizador ahora se obtiene siempre a través de la FK.

Sacar el campo de texto obligó a resolver un problema de permisos: mostrar el nombre del organizador en detalle.html necesita leer usuario (o federacion), pero sportana_publico no tiene acceso a esas tablas —correctamente, ya que usuario incluye password_hash—. En vez de otorgar ese acceso directo, se creó una vista (vista_organizador) que expone únicamente el nombre, y se le dio SELECT sobre la vista a sportana_publico, no sobre las tablas base.

```
-- Registrar participante y confirmar donde quedo cada dato --
$ curl -X POST ../procesar_registro.php \
  -d "nombre=Valentina&apellido=Suarez&correo=valen@test.com&usuario=valen_s&..."
{"ok":true,"id_usuario":1}

SELECT u.nombre_usuario, u.nombre, p.apellido
FROM usuario u JOIN participante p ON p.id_usuario=u.id_usuario;
nombre_usuario: valen_s  nombre: Valentina  apellido: Suarez

-- Registrar federacion: usuario.nombre se llena con el representante legal --
SELECT u.nombre, f.nombre AS nombre_federacion FROM usuario u JOIN federacion f ...;
nombre: Marcos Díaz  nombre_federacion: Fed Handball

-- Crear torneo SIN el campo organizador, y leerlo de vuelta --
$ curl -b cookies_fed.txt -X POST ../procesar_crear_torneo.php \
  -d "nombre=Copa Handball&deporte=Handball&formato=liga&..." (sin organizador=...)
{"ok":true,"id_torneo":1}

$ curl http://localhost/obtener_torneo.php?id=1
{..., "organizador": "Fed Handball", ...} <- viene de vista_organizador, no de texto
suelto
```

El nombre y apellido quedan en la tabla correcta, y el organizador de un torneo se lee siempre desde la FK real.

3.10 Relación con Ciberseguridad

Esta entrega se revisó específicamente contra las medidas que la segunda entrega de Ciberseguridad da por aplicadas, para que ambos documentos describan el mismo sistema real y no queden inconsistencias entre lo que un informe afirma y lo que el código efectivamente hace:

Medida (Ciberseguridad)	Implementación real en este backend
Política de contraseñas (8 car., mayúscula, minúscula, número, especial)	Validada en Usuario::registrar() antes de cualquier operación sobre la base de datos.
Hashing con bcrypt	password_hash() con PASSWORD_BCRYPT, verificado que el hash almacenado no es la contraseña en texto plano.
Bloqueo de 15 minutos tras 5 intentos fallidos	Tabla intento_login y verificación en Usuario::login() antes de validar credenciales.
Prepared Statements	Usadas en la totalidad de las consultas de Usuario.php.

Medida (Ciberseguridad)	Implementación real en este backend
Gestión segura de sesiones (expiración, regeneración de ID, cookies)	Configurado en procesar_login.php: expiración de 15 min, session_regenerate_id(), cookies HttpOnly y SameSite=Strict.
Manejo seguro de errores (mensajes genéricos, detalle en logs)	Los errores inesperados se capturan y registran con error_log(), mostrando al usuario un mensaje genérico.

Tabla 4. Correspondencia entre las medidas de Ciberseguridad y su implementación real en este backend.

3.11 Conclusión

Esta segunda entrega desarrolla la capa de datos y el backend de gestión de usuarios de Sportana: un modelo relacional normalizado de 16 tablas (con notación formal de generalización, entidades débiles y totalidad en el DER), un usuario de base de datos con permisos restringidos mediante DCL, y una integración funcional con PHP orientado a objetos que permite registrar e iniciar sesión tanto a participantes como a federaciones deportivas, crear torneos con sus reglas específicas a través del asistente real de tres pasos, inscribir participantes a un torneo por parte del organizador desde una página de detalle que ahora lee datos reales de la base, y comunicación asincrónica en JavaScript conectando esos formularios a sus respectivos endpoints sobre Apache. A diferencia de la documentación de especificación producida en Ingeniería de Software, todo lo descrito en este informe corresponde a código efectivamente escrito, ejecutado y probado —de punta a punta, contra una base de datos y un servidor Apache reales, aunque en un entorno de prueba y no en el servidor AlmaLinux propio del proyecto—, con la evidencia incluida en la sección 3.9. Queda pendiente el despliegue de este mismo código en el servidor real del equipo, y para la tercera entrega, la implementación completa del resto del sistema (torneos, equipos, rondas y resultados) sobre esta misma base, junto con su despliegue portable mediante Docker.

4. Bibliografía

- Oracle Corporation. (2024). *MySQL 8.0 Reference Manual — Data Definition, Manipulation, and Control Statements*. Oracle.
- The PHP Group. (2024). *PHP Manual — mysqli, password_hash, Object-Oriented Programming*.
- OWASP Foundation. (2023). *OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks*.
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377-387.

Hoja Testigo

Espacio reservado para correcciones y observaciones del cuerpo docente.

Firma del Profesor